1 - Primeira versão

Gestão de despesas - o que vamos construir primeiro

Esta é a primeira versão da vossa aplicação. Ainda não temos router nem base de dados - o objectivo é ter **uma página** a funcionar com interacção real, usando dados de teste.

1. O que aparece no ecrã

A aplicação tem uma única página dividida em três zonas visíveis:

Zona A - Resumo Três cards no topo: **Saldo actual**, **Total de receitas** e **Total de despesas**. Estes valores calculam-se automaticamente a partir da lista de transacções - não são introduzidos à mão.

Zona B - Formulário Um campo de texto para a descrição e um campo numérico para o valor. Um selector ou convenção para marcar se é **receita ou despesa**. Um botão para adicionar.

Zona C - Lista de transacções Uma lista scrollable com todas as transacções. Receitas aparecem a **verde**, despesas a **vermelho**. Cada entrada tem um botão para apagar.

2. Funcionalidades desta versão

Ver o saldo actual - calculado automaticamente: receitas menos despesas.

Ver receitas e despesas em separado - dois totais distintos, sempre actualizados.

Adicionar uma transacção - preencher o formulário e carregar no botão. A lista e os totais actualizam imediatamente.

Apagar uma transacção - botão em cada entrada da lista. Os totais recalculam sozinhos.

Distinguir receitas de despesas visualmente - verde para entradas positivas, vermelho para saídas.

3. Porquê dados de teste?

Nesta fase ainda não temos backend nem base de dados. Para não bloquear o desenvolvimento, vamos usar um ficheiro mockData.js com transacções já criadas - é a vossa "base de dados temporária".

Quando mais tarde aprendermos **useEffect** e ligarmos uma base de dados real, a única coisa que muda é de onde vêm os dados iniciais. **A estrutura dos componentes, o estado, as funções - tudo fica igual.** Estão a construir a fundação certa desde o início.

```
// mockData.js export const initialTransactions = [ { id: 1, description: "Salário de março", amount: 1850.00, type: "income", date: "2024-03-25", }, { id: 2, description: "Renda do apartamento", amount: 650.00, type: "expense", date: "2024-03-26", }, { id: 3, description: "Supermercado Pingo Doce", amount: 87.40, type: "expense", date: "2024-03-27", }, { id: 4, description: "Trabalho freelance", amount: 320.00, type: "income", date: "2024-03-28", }, { id: 5, description: "NOS - internet e telemóvel", amount: 45.00, type: "expense", date: "2024-03-29", }, { id: 6, description: "Jantar com clientes", amount: 112.80, type: "expense", date: "2024-03-30", }, { id: 7, description: "Reembolso de viagem", amount: 95.00, type: "income", date: "2024-03-31", }, { id: 8, description: "Ginásio Holmes Place", amount: 39.90, type: "expense", date: "2024-04-01", }, ]
```

4. Estrutura de ficheiros

```
src/ ├── App.jsx ├── mockData.js └── components/ ├── Summary.jsx ← os
três cards (saldo, receitas, despesas) ├── TransactionList.jsx ← a lista
despezas ├── TransactionItem.jsx ← um item individual (verde/vermelho)
└── AddTransaction.jsx ← o formulário
```

A aplicação está dividida assim:

ponto de entrada
App.jsx

O cérebro da aplicação. O estado (a lista de transacções) vive aqui. As funções de adicionar e apagar também. Importa todos os componentes e compõe a página.

dados de teste
mockData.js

Um array de transacções já preenchidas para começarem com dados reais no ecrã. Não é um componente - é só um ficheiro de dados que o App.jsx importa.

componente
components/Summary.jsx

Recebe os valores de saldo, receitas e despesas via props. Mostra os três cards. Não calcula nada - só apresenta o que recebe.

componente
components/AddTransaction.jsx

Tem o seu próprio estado local para os campos do formulário. Quando o utilizador carrega em adicionar, chama uma função que recebeu via props e passa os dados para cima (callback).

componente
components/TransactionList.jsx

Recebe a lista de transacções e a função de apagar via props. Usa map() para renderizar cada item. Pode ter scroll se a lista for longa.

componente
components/TransactionItem.jsx

Um único item da lista. Recebe a transacção e a função de apagar via props. Aplica a cor certa (verde ou vermelho) conforme o tipo. Tem o botão de apagar.

O que vamos aprender com esta versao

Esta versão foi desenhada para ser mais do que um exercício - cada funcionalidade que vais construir existe por uma razão técnica concreta. Aqui está o que vais praticar e onde vais sentir cada conceito a fazer sentido.

Estado no componente raiz, partilhado via props

A lista de transacções vive no `App.jsx` como estado. Sempre que adicionas ou apagas uma transacção, o React re-renderiza automaticamente tudo o que depende dessa lista - os totais, a lista, o saldo. Sem tocar no DOM, sem recarregar a página.

Vais perceber porque é que o estado vive no `App.jsx` e não dentro de cada componente: porque vários componentes (filhos) precisam de aceder ao mesmo dado.

> Onde vais aplicar: `App.jsx` - `const [transactions, setTransactions] = useState(initialTransactions)`

```
// Para importar mockdata e usar em App.jsx: import {
initialTransactions } from "./mockData"
```

useState local - estado local do formulário

O formulário de adicionar uma transacção tem os seus próprios campos - descrição e valor. Esse estado é **local**: só interessa ao formulário enquanto o utilizador está a preencher. Não precisa de viver no `App.jsx`.

Vais sentir a diferença entre estado que pertence a um componente e estado que pertence à aplicação toda. É uma das decisões mais importantes que tomamos em React.

Onde vais aplicar: `AddTransaction.jsx` - campos controlados com `onChange`

Props - passar dados para baixo

O `App.jsx` calcula o saldo, as receitas e as despesas - e passa esses valores via props para o `Summary.jsx`. Passa a lista de transacções para o `TransactionList.jsx`. Cada componente recebe exactamente o que precisa, nada mais.

Vais solidificar o hábito de pensar: **"este componente precisa de saber o quê para fazer o seu trabalho?"**

Onde vais aplicar: `Summary.jsx`, `TransactionList.jsx`, `TransactionItem.jsx`

Lifting state up - Passar funções para cima

O formulário de adicionar e o botão de apagar vivem em componentes filhos - mas quem controla a lista é o `App.jsx`. A solução é passar as funções `handleAdd` e `handleDelete` como props para baixo, para que os filhos as possam chamar quando precisam.

Este é o padrão mais importante desta versão. Vais perceber na prática porque é que **o estado sobe, os eventos descem**.

map() e filter() - Manipular arrays em estado

Vais usar `map()` para renderizar a lista de transacções - cada objecto do array torna-se um componente no ecrã. Vais usar `filter()` para apagar uma transacção (devolver um novo array sem o item em causa) e para calcular os totais de receitas e despesas.

Estes métodos já conhecem da aula - aqui vão usá-los num contexto real, com dados que mudam e um ecrã que reage.

Onde vais aplicar: `TransactionList.jsx` para renderizar, `App.jsx` para apagar e calcular totais

Renderização condicional - Mostrar coisas diferentes conforme os dados

Cada transacção é receita ou despesa - e o ecrã tem de reflectir isso com cores diferentes. Vais aplicar lógica condicional no JSX para aplicar a classe de cor certa a cada item, com base no campo `type` do objecto.

É um exemplo simples mas muito concreto de como o React torna a UI **um reflexo directo dos dados**.

Onde vais aplicar: `TransactionItem.jsx` - cor verde se `type === "income"`, vermelho se `type === "expense"`